



```
In [1]: import pandas as pd
import numpy as np
import os
import cv2
import random
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
import warnings
warnings.filterwarnings('ignore')

plt.rcParams['figure.figsize'] = (12, 8)
sns.set_style("whitegrid")
```

```
In [2]: labels_path = "train_labels_.csv"
labels = pd.read_csv(labels_path)

print(f"Всего записей: {len(labels)}")
print(f"Уникальных изображений: {labels['filename'].nunique()}")
print("\nСтруктура датасета:")
print(labels.info())
print("\nПервые 5 записей:")
print(labels.head())
```

Всего записей: 173

Уникальных изображений: 160

Структура датасета:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 173 entries, 0 to 172

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	filename	173 non-null	object
1	width	173 non-null	int64
2	height	173 non-null	int64
3	class	173 non-null	object
4	xmin	173 non-null	int64
5	ymin	173 non-null	int64
6	xmax	173 non-null	int64
7	ymax	173 non-null	int64

dtypes: int64(6), object(2)

memory usage: 10.9+ KB

None

Первые 5 записей:

	filename	width	height	class	xmin	ymin	xmax	ymax
0	raccoon-17.jpg	259	194	raccoon	95	60	167	118
1	raccoon-11.jpg	660	432	raccoon	3	1	461	431
2	raccoon-63.jpg	600	400	raccoon	74	107	280	290
3	raccoon-63.jpg	600	400	raccoon	227	93	403	298
4	raccoon-60.jpg	273	185	raccoon	58	33	197	127

```
In [5]: analysis_df = labels.copy()
```

```

analysis_df['bbox_width'] = analysis_df['xmax'] - analysis_df['xmin']
analysis_df['bbox_height'] = analysis_df['ymax'] - analysis_df['ymin']
analysis_df['bbox_area'] = analysis_df['bbox_width'] * analysis_df['bbox_height']
analysis_df['aspect_ratio'] = analysis_df['bbox_width'] / analysis_df['bbox_height']
analysis_df['image_area'] = analysis_df['width'] * analysis_df['height']
analysis_df['relative_area'] = analysis_df['bbox_area'] / analysis_df['image_area']

objects_per_image = analysis_df.groupby('filename').size()

print(f"Максимальное количество ентов на изображении: {objects_per_image.max()}")
print(f"Минимальное количество ентов на изображении: {objects_per_image.min()}")
print(f"Среднее количество ентов на изображении: {objects_per_image.mean():.2}")
print(f"Медианное количество ентов на изображении: {objects_per_image.median()}")

fig, axes = plt.subplots(2, 3, figsize=(15, 10))

axes[0, 0].hist(objects_per_image.values, bins=20, edgecolor='black', alpha=0.5)
axes[0, 0].set_xlabel('Количество объектов')
axes[0, 0].set_ylabel('Частота')
axes[0, 0].set_title('Распределение объектов по изображениям')
axes[0, 0].axvline(objects_per_image.mean(), color='red', linestyle='--', label=f'Среднее: {objects_per_image.mean():.2}')
axes[0, 0].legend()

axes[0, 1].scatter(analysis_df['bbox_width'], analysis_df['bbox_height'], alpha=0.5)
axes[0, 1].set_xlabel('Ширина bbox')
axes[0, 1].set_ylabel('Высота bbox')
axes[0, 1].set_title('Размеры bounding boxes')

axes[0, 2].hist(analysis_df['aspect_ratio'], bins=50, edgecolor='black', alpha=0.5)
axes[0, 2].set_xlabel('Соотношение сторон (width/height)')
axes[0, 2].set_ylabel('Частота')
axes[0, 2].set_title('Распределение соотношений сторон')
axes[0, 2].axvline(1, color='red', linestyle='--', label='Квадрат')

axes[1, 0].hist(analysis_df['relative_area'], bins=50, edgecolor='black', alpha=0.5)
axes[1, 0].set_xlabel('Относительная площадь объекта')
axes[1, 0].set_ylabel('Частота')
axes[1, 0].set_title('Относительный размер объектов')

unique_sizes = analysis_df[['width', 'height']].drop_duplicates()
axes[1, 1].scatter(unique_sizes['width'], unique_sizes['height'], alpha=0.7)
axes[1, 1].set_xlabel('Ширина изображения')
axes[1, 1].set_ylabel('Высота изображения')
axes[1, 1].set_title('Размеры изображений в датасете')

```

```

analysis_df['center_x'] = (analysis_df['xmin'] + analysis_df['xmax']) / 2 / ar
analysis_df['center_y'] = (analysis_df['ymin'] + analysis_df['ymax']) / 2 / ar
scatter = axes[1, 2].scatter(analysis_df['center_x'], analysis_df['center_y'],
                             alpha=0.5, c=analysis_df['relative_area'], cmap='v
axes[1, 2].set_xlabel('Нормированная координата X центра')
axes[1, 2].set_ylabel('Нормированная координата Y центра')
axes[1, 2].set_title('Положение объектов на изображениях')
plt.colorbar(scatter, ax=axes[1, 2], label='Относительная площадь')

plt.tight_layout()
plt.show()

correlation_matrix = analysis_df[['bbox_width', 'bbox_height', 'bbox_area',
                                  'aspect_ratio', 'relative_area']].corr()

print(correlation_matrix)

plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0)
plt.title('Корреляционная матрица параметров bounding boxes')
plt.show()

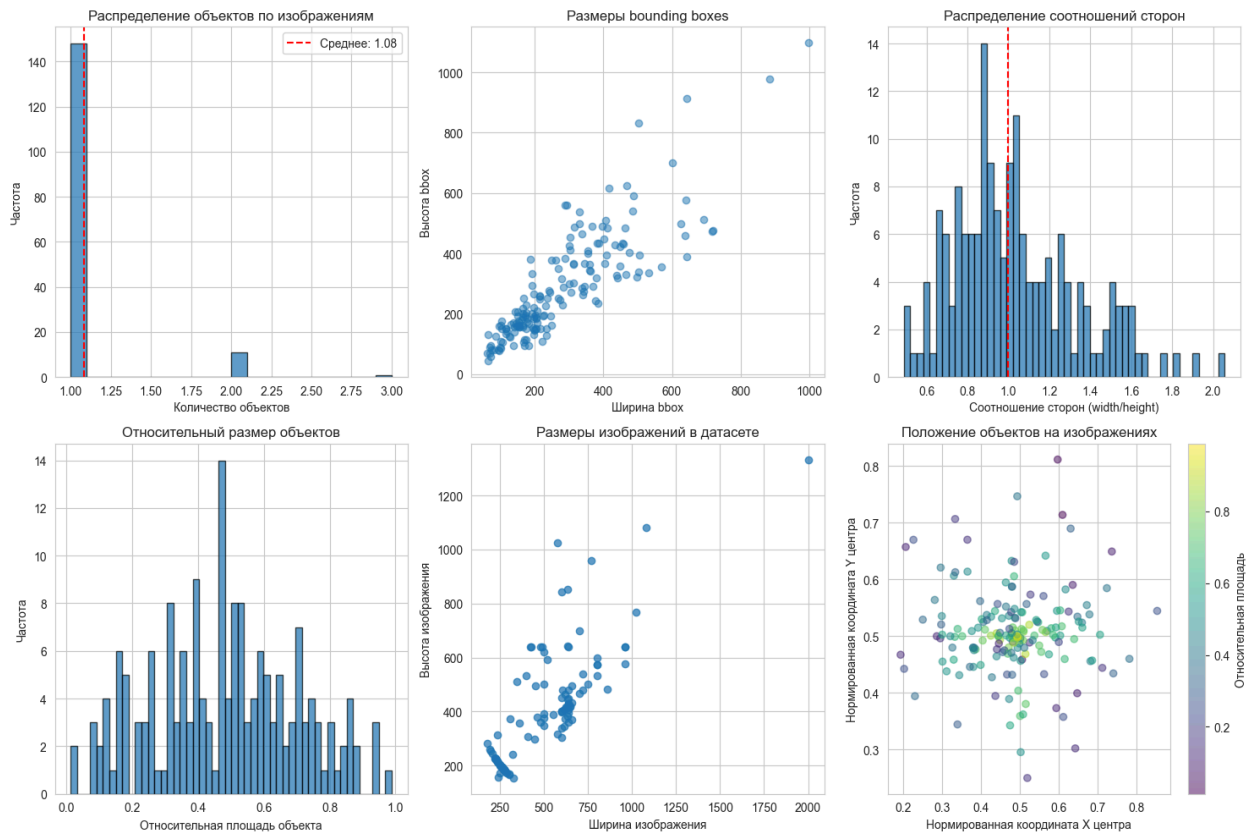
```

Максимальное количество енотов на изображении: 3

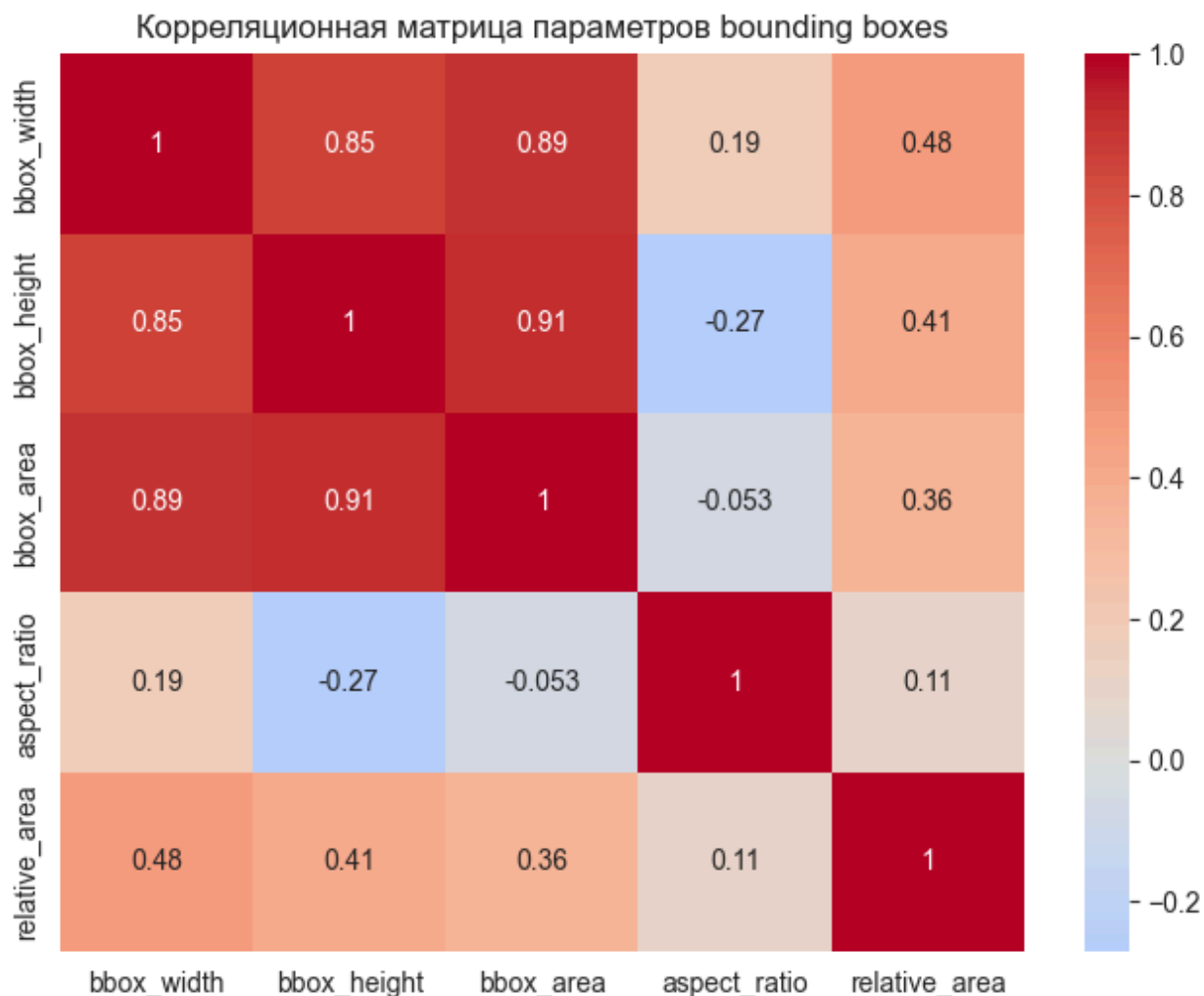
Минимальное количество енотов на изображении: 1

Среднее количество енотов на изображении: 1.08

Медианное количество енотов на изображении: 1.0



	bbox_width	bbox_height	bbox_area	aspect_ratio	relative_area
bbox_width	1.000000	0.854330	0.892375	0.185465	0.484390
bbox_height	0.854330	1.000000	0.911832	-0.271289	0.413149
bbox_area	0.892375	0.911832	1.000000	-0.052837	0.357662
aspect_ratio	0.185465	-0.271289	-0.052837	1.000000	0.105038
relative_area	0.484390	0.413149	0.357662	0.105038	1.000000



```
In [8]: def convert_to_yolo_format(csv_path, image_dir, output_dir):

    os.makedirs(output_dir, exist_ok=True)

    df = pd.read_csv(csv_path)

    class_mapping = {"raccoon": 0}

    total_images = 0
    total_objects = 0

    grouped = df.groupby("filename")

    for filename, group in tqdm(grouped, desc="Обработка изображений"):
        img_path = os.path.join(image_dir, filename)
```

```

if not os.path.exists(img_path):
    print(f"Предупреждение: {img_path} не найден")
    continue

img = cv2.imread(img_path)
if img is None:
    print(f"Ошибка загрузки: {img_path}")
    continue

h, w = img.shape[:2]
yolo_lines = []

for _, row in group.iterrows():
    class_id = class_mapping.get(row["class"], 0)

    x_center = ((row["xmin"] + row["xmax"]) / 2) / w
    y_center = ((row["ymin"] + row["ymax"]) / 2) / h
    bbox_width = (row["xmax"] - row["xmin"]) / w
    bbox_height = (row["ymax"] - row["ymin"]) / h

    if (0 <= x_center <= 1 and 0 <= y_center <= 1 and
        0 < bbox_width <= 1 and 0 < bbox_height <= 1):
        yolo_lines.append(f"{class_id} {x_center:.6f} {y_center:.6f} {
            total_objects += 1
    else:
        print(f"Некорректные координаты в {filename}: {row}")

if yolo_lines:
    txt_filename = filename.replace(".jpg", ".txt")
    txt_path = os.path.join(output_dir, txt_filename)

    with open(txt_path, "w") as f:
        f.write("\n".join(yolo_lines))

    total_images += 1

print(f"\nРезультаты конвертации:")
print(f"    Успешно обработано изображений: {total_images}")
print(f"    Всего объектов: {total_objects}")

return total_images, total_objects

image_dir = "Raccoon Images/images"
output_yolo_dir = "labels_yolo_v2"

yolo_images, yolo_objects = convert_to_yolo_format(labels_path, image_dir, out

```

Обработка изображений: 100%|██████████| 160/160 [00:01<00:00, 129.73it/s]

Результаты конвертации:

Успешно обработано изображений: 160

Всего объектов: 173

```

In [29]: def visualize_annotations(image_dir, label_dir, num_samples=5):

    image_files = [f for f in os.listdir(image_dir) if f.endswith('.jpg')]

    selected_images = random.sample(image_files, min(num_samples, len(image_files)))

    fig, axes = plt.subplots(2, 3, figsize=(15, 10))
    axes = axes.flatten()

    for idx, img_name in enumerate(selected_images[:6]):
        img_path = os.path.join(image_dir, img_name)
        img = cv2.imread(img_path)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        txt_path = os.path.join(label_dir, img_name.replace(".jpg", ".txt"))

        if os.path.exists(txt_path):
            with open(txt_path, 'r') as f:
                lines = f.readlines()

            h, w = img.shape[:2]
            for line in lines:
                class_id, x_center, y_center, bbox_w, bbox_h = map(float, line.split())

                x_center_px = int(x_center * w)
                y_center_px = int(y_center * h)
                bbox_w_px = int(bbox_w * w)
                bbox_h_px = int(bbox_h * h)

                x_min = x_center_px - bbox_w_px // 2
                y_min = y_center_px - bbox_h_px // 2
                x_max = x_center_px + bbox_w_px // 2
                y_max = y_center_px + bbox_h_px // 2

                cv2.rectangle(img, (x_min, y_min), (x_max, y_max), (255, 0, 0))
                cv2.putText(img, 'raccoon', (x_min, y_min-10),
                           cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 2)

            axes[idx].imshow(img)
            axes[idx].set_title(f'{img_name}\nОбъектов: {len(lines)} if os.path.exists(txt_path):')
            axes[idx].axis('off')

    for idx in range(len(selected_images[:6]), 6):
        axes[idx].axis('off')

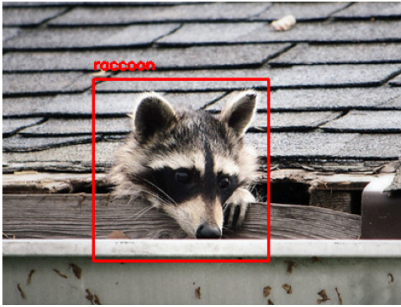
    plt.suptitle('Примеры изображений с bounding boxes', fontsize=16)
    plt.tight_layout()
    plt.show()

visualize_annotations(image_dir, output_yolo_dir)

```

Примеры изображений с bounding boxes

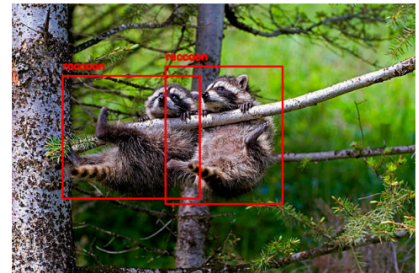
гассооп-7.jpg
Объектов: 1



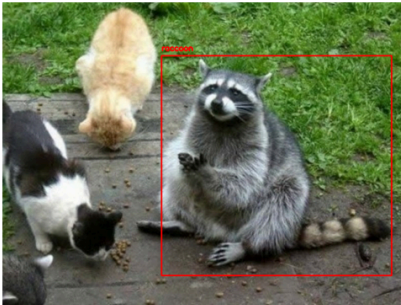
гассооп-143.jpg
Объектов: 0



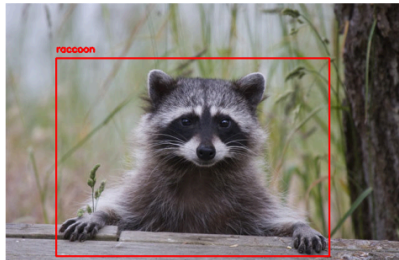
гассооп-63.jpg
Объектов: 2



гассооп-51.jpg
Объектов: 1



гассооп-1.jpg
Объектов: 1



In [30]: `%pip install albumentations`

Requirement already satisfied: albumations in c:\users\anyab\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (2.0.8)

Requirement already satisfied: numpy>=1.24.4 in c:\users\anyab\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from albumations) (2.2.3)

Requirement already satisfied: scipy>=1.10.0 in c:\users\anyab\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from albumations) (1.15.2)

Requirement already satisfied: PyYAML in c:\users\anyab\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from albumations) (6.0.2)

Requirement already satisfied: pydantic>=2.9.2 in c:\users\anyab\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from albumations) (2.12.5)

Requirement already satisfied: albucore==0.0.24 in c:\users\anyab\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from albumations) (0.0.24)

Requirement already satisfied: opencv-python-headless>=4.9.0.80 in c:\users\anyab\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from albumations) (4.12.0.88)

Requirement already satisfied: stringzilla>=3.10.4 in c:\users\anyab\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from albucore==0.0.24->albumations) (4.4.0)

Requirement already satisfied: simsimd>=5.9.2 in c:\users\anyab\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from albucore==0.0.24->albumations) (6.5.3)

Requirement already satisfied: annotated-types>=0.6.0 in c:\users\anyab\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from pydantic>=2.9.2->albumations) (0.7.0)

Requirement already satisfied: pydantic-core==2.41.5 in c:\users\anyab\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from pydantic>=2.9.2->albumations) (2.41.5)

Requirement already satisfied: typing-extensions>=4.14.1 in c:\users\anyab\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from pydantic>=2.9.2->albumations) (4.15.0)

Requirement already satisfied: typing-inspection>=0.4.2 in c:\users\anyab\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from pydantic>=2.9.2->albumations) (0.4.2)

Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 24.0 -> 25.3

[notice] To update, run: C:\Users\anyab\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip

```
In [40]: import cv2
import numpy as np
```



```

import random
import os
from tqdm import tqdm

class OpenCVAugmentor:

    def __init__(self, aug_type='standard'):
        self.aug_type = aug_type

    def create_pipeline(self):
        if self.aug_type == 'light':
            return self._light_pipeline
        elif self.aug_type == 'standard':
            return self._standard_pipeline
        elif self.aug_type == 'heavy':
            return self._heavy_pipeline
        else:
            return lambda img, bboxes: (img, bboxes)

    def _light_pipeline(self, image, bboxes):
        aug_image = image.copy()
        aug_bboxes = [bbox.copy() for bbox in bboxes]

        if random.random() < 0.3:
            aug_image, aug_bboxes = self._horizontal_flip(aug_image, aug_bboxes)

        if random.random() < 0.2:
            aug_image = self._adjust_brightness_contrast(aug_image)

        return aug_image, aug_bboxes

    def _standard_pipeline(self, image, bboxes):
        aug_image = image.copy()
        aug_bboxes = [bbox.copy() for bbox in bboxes]

        if random.random() < 0.5:
            aug_image, aug_bboxes = self._horizontal_flip(aug_image, aug_bboxes)

        if random.random() < 0.4:
            aug_image = self._adjust_brightness_contrast(aug_image)

        if random.random() < 0.3:
            angle = random.uniform(-15, 15)
            aug_image = self._rotate(aug_image, angle)

        if random.random() < 0.2:
            aug_image = self._apply_blur(aug_image)

        if random.random() < 0.1:
            aug_image = self._add_gaussian_noise(aug_image)

        if random.random() < 0.3:

```

```

        aug_image = self._adjust_hue_saturation(aug_image)

    return aug_image, aug_bboxes

def _heavy_pipeline(self, image, bboxes):
    aug_image = image.copy()
    aug_bboxes = [bbox.copy() for bbox in bboxes]

    if random.random() < 0.5:
        aug_image, aug_bboxes = self._horizontal_flip(aug_image, aug_bboxes)

    if random.random() < 0.1:
        aug_image = cv2.flip(aug_image, 0)

    if random.random() < 0.5:
        aug_image = self._adjust_brightness_contrast(aug_image, strong=True)

    if random.random() < 0.4:
        angle = random.uniform(-30, 30)
        aug_image = self._rotate(aug_image, angle)

    if random.random() < 0.3:
        scale = random.uniform(0.7, 1.3)
        h, w = aug_image.shape[:2]
        aug_image = cv2.resize(aug_image, (int(w * scale), int(h * scale)))

    if random.random() < 0.3:
        blur_type = random.choice(['gaussian', 'median', 'motion'])
        if blur_type == 'gaussian':
            aug_image = cv2.GaussianBlur(aug_image, (5, 5), 0)
        elif blur_type == 'median':
            aug_image = cv2.medianBlur(aug_image, 5)
        elif blur_type == 'motion':
            kernel_size = random.choice([3, 5, 7])
            kernel = np.zeros((kernel_size, kernel_size))
            kernel[int((kernel_size-1)/2), :] = np.ones(kernel_size)
            kernel = kernel / kernel_size
            aug_image = cv2.filter2D(aug_image, -1, kernel)

    if random.random() < 0.2:
        aug_image = self._add_gaussian_noise(aug_image, strong=True)

    if random.random() < 0.4:
        aug_image = self._adjust_hue_saturation(aug_image, strong=True)

    if random.random() < 0.2:
        aug_image = self._apply_cutout(aug_image)

    return aug_image, aug_bboxes

def _horizontal_flip(self, image, bboxes):
    flipped = cv2.flip(image, 1)

```

```

        new_bboxes = []
        for bbox in bboxes:
            x_center, y_center, width, height = bbox
            new_bboxes.append([1.0 - x_center, y_center, width, height])

        return flipped, new_bboxes

def _adjust_brightness_contrast(self, image, strong=False):
    if strong:
        alpha = random.uniform(0.5, 1.5)
        beta = random.uniform(-50, 50)
    else:
        alpha = random.uniform(0.8, 1.2)
        beta = random.uniform(-30, 30)

    img_float = image.astype(np.float32)
    adjusted = alpha * img_float + beta
    adjusted = np.clip(adjusted, 0, 255).astype(np.uint8)
    return adjusted

def _rotate(self, image, angle):

    h, w = image.shape[:2]
    center = (w // 2, h // 2)

    M = cv2.getRotationMatrix2D(center, angle, 1.0)

    cos = np.abs(M[0, 0])
    sin = np.abs(M[0, 1])
    new_w = int((h * sin) + (w * cos))
    new_h = int((h * cos) + (w * sin))

    M[0, 2] += (new_w / 2) - center[0]
    M[1, 2] += (new_h / 2) - center[1]

    rotated = cv2.warpAffine(image, M, (new_w, new_h))
    return rotated

def _apply_blur(self, image):
    kernel_size = random.choice([3, 5])
    blurred = cv2.GaussianBlur(image, (kernel_size, kernel_size), 0)
    return blurred

def _add_gaussian_noise(self, image, strong=False):
    h, w, c = image.shape

    if strong:
        noise = np.random.normal(0, 30, (h, w, c)).astype(np.float32)
    else:
        noise = np.random.normal(0, 15, (h, w, c)).astype(np.float32)

```

```

noisy = image.astype(np.float32) + noise
noisy = np.clip(noisy, 0, 255).astype(np.uint8)
return noisy

def _adjust_hue_saturation(self, image, strong=False):

    hsv = cv2.cvtColor(image, cv2.COLOR_RGB2HSV).astype(np.float32)

    if strong:
        hue_shift = random.uniform(-20, 20)
        sat_scale = random.uniform(0.7, 1.3)
        val_scale = random.uniform(0.7, 1.3)
    else:
        hue_shift = random.uniform(-10, 10)
        sat_scale = random.uniform(0.8, 1.2)
        val_scale = random.uniform(0.8, 1.2)

    hsv[:, :, 0] = (hsv[:, :, 0] + hue_shift) % 180
    hsv[:, :, 1] = np.clip(hsv[:, :, 1] * sat_scale, 0, 255)
    hsv[:, :, 2] = np.clip(hsv[:, :, 2] * val_scale, 0, 255)

    adjusted = cv2.cvtColor(hsv.astype(np.uint8), cv2.COLOR_HSV2RGB)
    return adjusted

def _apply_cutout(self, image, num_holes=5, max_h_size=30, max_w_size=30):

    aug_image = image.copy()
    h, w, _ = aug_image.shape

    for _ in range(num_holes):

        x1 = random.randint(0, w - max_w_size)
        y1 = random.randint(0, h - max_h_size)
        x2 = x1 + random.randint(10, max_w_size)
        y2 = y1 + random.randint(10, max_h_size)

        aug_image[y1:y2, x1:x2] = 0

    return aug_image

def apply(self, image, bboxes):
    pipeline = self.create_pipeline()
    return pipeline(image, bboxes)

def create_augmentation_pipeline(aug_type='standard'):

    return OpenCVAugmentor(aug_type=aug_type)

def apply_augmentation(image_dir, label_dir, output_img_dir, output_label_dir,
                        num_augmentations=2, aug_type='standard'):

```

```

os.makedirs(output_img_dir, exist_ok=True)
os.makedirs(output_label_dir, exist_ok=True)

augmentor = create_augmentation_pipeline(aug_type)

total_original = 0
total_augmented = 0

image_files = [f for f in os.listdir(image_dir) if f.endswith('.jpg')]

print(f"Начало аугментации (тип: {aug_type})...")

for img_file in tqdm(image_files, desc="Аугментация"):

    if img_file.startswith('aug_'):
        continue

    img_path = os.path.join(image_dir, img_file)
    label_path = os.path.join(label_dir, img_file.replace('.jpg', '.txt'))

    if not os.path.exists(label_path):
        continue

    image = cv2.imread(img_path)
    if image is None:
        continue

    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    h, w = image.shape[:2]

    with open(label_path, 'r') as f:
        lines = f.readlines()

    bboxes = []
    class_labels = []
    for line in lines:
        class_id, x_center, y_center, bbox_w, bbox_h = map(float, line.strip().split())
        bboxes.append([x_center, y_center, bbox_w, bbox_h])
        class_labels.append(int(class_id))

    total_original += 1

    cv2.imwrite(os.path.join(output_img_dir, img_file),
                cv2.cvtColor(image, cv2.COLOR_RGB2BGR))

    with open(os.path.join(output_label_dir, img_file.replace('.jpg', '.txt')), 'w') as f:
        f.writelines(lines)

    for aug_idx in range(num_augmentations):
        try:

```

```

        aug_image, aug_bboxes = augmentor.apply(image, bboxes)

        aug_name = f"aug_{aug_idx}_{img_file}"

        cv2.imwrite(
            os.path.join(output_img_dir, aug_name),
            cv2.cvtColor(aug_image, cv2.COLOR_RGB2BGR)
        )

        with open(os.path.join(output_label_dir, aug_name.replace('.jpg', '.txt')), 'a') as f:
            for label, bbox in zip(class_labels, aug_bboxes):
                f.write(f"{label} {bbox[0]:.6f} {bbox[1]:.6f} {bbox[2]:.6f} {bbox[3]:.6f}\n")

        total_augmented += 1

    except Exception as e:
        print(f"Ошибка при аугментации {img_file}: {e}")
        continue

print(f"\nРезультаты аугментации:")
print(f"    Оригинальных изображений: {total_original}")
print(f"    Аугментированных изображений: {total_augmented}")
print(f"    Всего изображений после аугментации: {total_original + total_augmented}")
print(f"    Коэффициент увеличения датасета: {(total_augmented/total_original):.2f}")

return total_original, total_augmented

def visualize_augmentation_results(image_dir, label_dir, num_samples=3):

    import matplotlib.pyplot as plt

    aug_files = [f for f in os.listdir(image_dir) if f.startswith('aug_') and f.endswith('.jpg')]

    if not aug_files:
        print("Нет аугментированных файлов для визуализации")
        return

    selected_files = random.sample(aug_files, min(num_samples, len(aug_files)))

    fig, axes = plt.subplots(1, num_samples, figsize=(15, 5))

    for idx, img_file in enumerate(selected_files):

        img_path = os.path.join(image_dir, img_file)
        img = cv2.imread(img_path)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        label_path = os.path.join(label_dir, img_file.replace('.jpg', '.txt'))

```

```

if os.path.exists(label_path):
    with open(label_path, 'r') as f:
        lines = f.readlines()

    h, w = img.shape[:2]
    for line in lines:
        parts = line.strip().split()
        if len(parts) == 5:
            class_id, x_center, y_center, bbox_w, bbox_h = map(float,

            x_center_px = int(x_center * w)
            y_center_px = int(y_center * h)
            bbox_w_px = int(bbox_w * w)
            bbox_h_px = int(bbox_h * h)

            x_min = max(0, x_center_px - bbox_w_px // 2)
            y_min = max(0, y_center_px - bbox_h_px // 2)
            x_max = min(w, x_center_px + bbox_w_px // 2)
            y_max = min(h, y_center_px + bbox_h_px // 2)

            cv2.rectangle(img, (x_min, y_min), (x_max, y_max), (0, 255
            cv2.putText(img, 'raccoon', (x_min, max(20, y_min-10)),
                           cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

    if num_samples == 1:
        axes.imshow(img)
        axes.set_title(f'{img_file}\nОбъектов: {len(lines)} if os.path.exists
        axes.axis('off')
    else:
        axes[idx].imshow(img)
        axes[idx].set_title(f'{img_file}\nОбъектов: {len(lines)} if os.path
        axes[idx].axis('off')

plt.suptitle('Примеры аугментированных изображений', fontsize=14)
plt.tight_layout()
plt.show()

image_dir = "Raccoon Images/images"
output_yolo_dir = "labels_yolo_v2"

output_aug_img_dir = "augmented_images_v2"
output_aug_label_dir = "augmented_labels_v2"

orig_count, aug_count = apply_augmentation(
    image_dir=image_dir,
    label_dir=output_yolo_dir,
    output_img_dir=output_aug_img_dir,
    output_label_dir=output_aug_label_dir,
    num_augmentations=3,

```



```

    aug_type='standard'
)

visualize_augmentation_results(output_aug_img_dir, output_aug_label_dir, num_s

def create_train_val_split(aug_img_dir, aug_label_dir, train_ratio=0.8, seed=4

    random.seed(seed)

    all_images = [f for f in os.listdir(aug_img_dir) if f.endswith('.jpg')]
    random.shuffle(all_images)

    split_idx = int(len(all_images) * train_ratio)
    train_images = all_images[:split_idx]
    val_images = all_images[split_idx:]

    train_img_dir = "dataset_v2/train/images"
    train_label_dir = "dataset_v2/train/labels"
    val_img_dir = "dataset_v2/val/images"
    val_label_dir = "dataset_v2/val/labels"

    for directory in [train_img_dir, train_label_dir, val_img_dir, val_label_dir]:
        os.makedirs(directory, exist_ok=True)

    print("\nСоздание train/val разделения...")

    for img_file in tqdm(all_images, desc="Копирование файлов"):
        if img_file in train_images:
            dst_img_dir = train_img_dir
            dst_label_dir = train_label_dir
        else:
            dst_img_dir = val_img_dir
            dst_label_dir = val_label_dir

        src_img_path = os.path.join(aug_img_dir, img_file)
        dst_img_path = os.path.join(dst_img_dir, img_file)

        img = cv2.imread(src_img_path)
        cv2.imwrite(dst_img_path, img)

        label_file = img_file.replace('.jpg', '.txt')
        src_label_path = os.path.join(aug_label_dir, label_file)
        dst_label_path = os.path.join(dst_label_dir, label_file)

        if os.path.exists(src_label_path):
            with open(src_label_path, 'r') as src, open(dst_label_path, 'w') as dst:
                dst.write(src.read())

    with open("dataset_v2/train.txt", "w") as f:

```

```

        for img_file in train_images:
            f.write(f"./images/{img_file}\n")

    with open("dataset_v2/val.txt", "w") as f:
        for img_file in val_images:
            f.write(f"./images/{img_file}\n")

    with open("dataset_v2/classes.names", "w") as f:
        f.write("raccoon\n")

    with open("dataset_v2/raccoon.yaml", "w") as f:
        f.write(f"""# Raccoon Detection Dataset
path: ./dataset_v2
train: train/images
val: val/images

# Classes
nc: 1
names: ['raccoon']
""")

    print(f"\nTrain изображений: {len(train_images)}")
    print(f"Val изображений: {len(val_images)}")

    return len(train_images), len(val_images)

train_count, val_count = create_train_val_split(output_aug_img_dir, output_aug

print(f"\n1. АУГМЕНТАЦИЯ:")
print(f"    - Оригинальных изображений: {orig_count}")
print(f"    - Аугментированных изображений: {aug_count}")
print(f"    - Всего изображений: {orig_count + aug_count}")
print(f"    - Коэффициент увеличения: {aug_count/orig_count:.2f}x")

print(f"\n2. РАЗДЕЛЕНИЕ НА TRAIN/VAL:")
print(f"    - Train изображений: {train_count} ({train_count/(train_count+val_c
print(f"    - Val изображений: {val_count} ({val_count/(train_count+val_count)*

```

Начало аугментации (тип: standard)...

Аугментация: 100%|██████████| 200/200 [00:08<00:00, 22.83it/s]

Результаты аугментации:

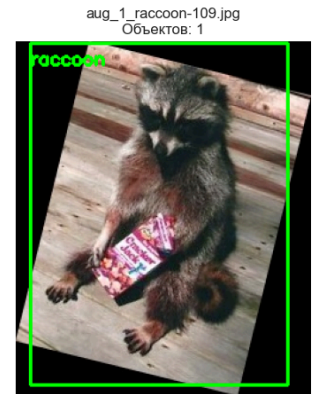
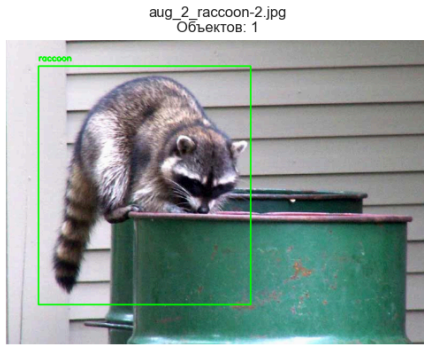
Оригинальных изображений: 160

Аугментированных изображений: 480

Всего изображений после аугментации: 640

Коэффициент увеличения датасета: 3.0x

Примеры аугментированных изображений



Создание train/val разделения...

Копирование файлов: 100% | ██████████ | 640/640 [00:16<00:00, 39.50it/s]

Train изображений: 512

Val изображений: 128

1. АУГМЕНТАЦИЯ:

- Оригинальных изображений: 160
- Аугментированных изображений: 480
- Всего изображений: 640
- Коэффициент увеличения: 3.00x

2. РАЗДЕЛЕНИЕ НА TRAIN/VAL:

- Train изображений: 512 (80.0%)
- Val изображений: 128 (20.0%)

```
In [43]: def visualize_augmentations(aug_img_dir, aug_label_dir, num_samples=3):

    aug_files = [f for f in os.listdir(aug_img_dir) if f.startswith('aug_') and
                  f.endswith('.jpg')]

    if not aug_files:
        print("Нет аугментированных файлов для визуализации")
        return

    selected_aug = random.sample(aug_files, min(num_samples * 3, len(aug_files)))

    fig, axes = plt.subplots(num_samples, 3, figsize=(15, 5 * num_samples))

    for row in range(num_samples):
        for col in range(3):
            idx = row * 3 + col
            if idx < len(selected_aug):
                img_name = selected_aug[idx]

                img_path = os.path.join(aug_img_dir, img_name)
                img = cv2.imread(img_path)
                img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

                txt_path = os.path.join(aug_label_dir, img_name.replace('.jpg', '.txt'))
```

```

if os.path.exists(txt_path):
    with open(txt_path, 'r') as f:
        lines = f.readlines()

    h, w = img.shape[:2]
    for line in lines:
        class_id, x_center, y_center, bbox_w, bbox_h = map(float, line.split())

        x_center_px = int(x_center * w)
        y_center_px = int(y_center * h)
        bbox_w_px = int(bbox_w * w)
        bbox_h_px = int(bbox_h * h)

        x_min = max(0, x_center_px - bbox_w_px // 2)
        y_min = max(0, y_center_px - bbox_h_px // 2)
        x_max = min(w, x_center_px + bbox_w_px // 2)
        y_max = min(h, y_center_px + bbox_h_px // 2)

        cv2.rectangle(img, (x_min, y_min), (x_max, y_max), (0,
        cv2.putText(img, 'raccoon', (x_min, y_min-10),
                      cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0),

    axes[row, col].imshow(img)
    axes[row, col].set_title(f'{img_name}\nОбъектов: {len(lines)}')
    axes[row, col].axis('off')
else:
    axes[row, col].axis('off')

plt.suptitle('Примеры аугментированных изображений', fontsize=16)
plt.tight_layout()
plt.show()

```

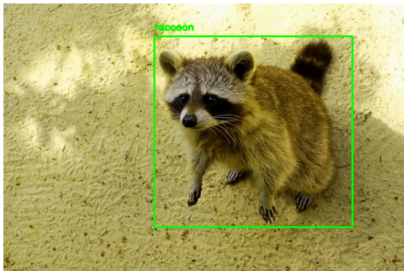
```

visualize_augmentations(output_aug_img_dir, output_aug_label_dir, num_samples=

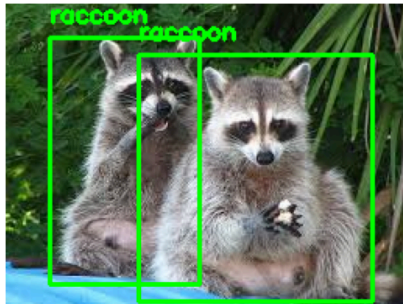
```

Примеры аугментированных изображений

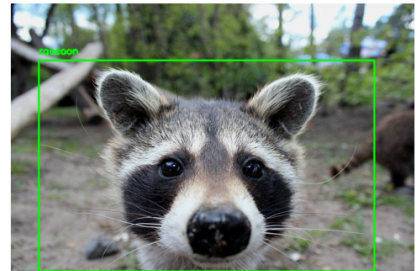
aug_0_raccoon-101.jpg
Объектов: 1



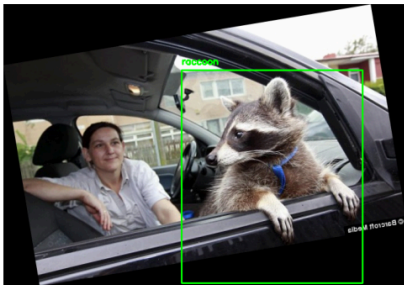
aug_1_raccoon-12.jpg
Объектов: 2



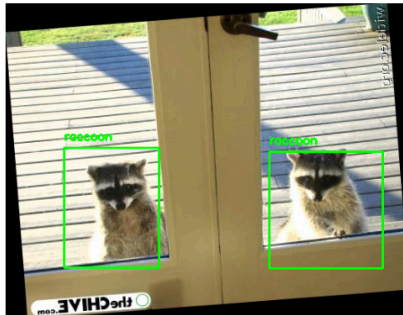
aug_0_raccoon-90.jpg
Объектов: 1



aug_2_raccoon-175.jpg
Объектов: 1



aug_1_raccoon-148.jpg
Объектов: 2



aug_2_raccoon-52.jpg
Объектов: 1

